有方机器人产品用户手册

——PS2无线手柄





官方淘宝：https://item.taobao.com/item.htm?id=653189570774

使用说明

感谢您使用有方机器人出品的PS2遥控手柄。

使用说明的作用在于辅助使用者快速了解PS2遥控手柄的硬件连接方式、通讯协议以及手柄的使用说明。

以下是使用本产品的一些注意事项，望知晓：

特殊说明

本手册版权归有方机器人所有，非经有方机器人授权同意（书面形式），任何单位及个人不得擅自摘取手册部分及全部内容用于商业用途，违者将追究其法律责任。

版本信息说明

版本	变更内容	负责人	审核人	时间
V1.0.0	[新增]建立文档	陈梦婷	敖继渊	2023.12.28

目录

有方机器人产品用户手册 1
使用说明 2
版本信息说明 2
一、 PS2 遥控器版本说明 4
二、 接收器及转接板引脚说明 5
2.1 接收器硬件说明 5
2.2 转接板硬件说明 6
2.3 转接板连接主机说明 6
三、 ps2 遥控手柄通讯协议 7
四、 ps2 遥控手柄使用说明 9
4.1 手柄模式说明 9
4.2 手柄的使用、连接配对说明 9
五、 程序说明 10
5.1 手柄 GPIO 初始化函数 10
5.2 手柄配置函数 11
5.3 手柄配置初始化 13
5.4 发送、接收数据函数 14
5.5 判断手柄模式函数 15
5.6 判断按键状态函数 16
5.7 摇杆值读取函数 16

一、PS2 遥控器版本说明

ps2 手柄由手柄、接收器、转接板三部分部分组成，如下图所示：



其中手柄主要负责发送按键信息，接收器与转接板安装在一起再与单片机相连，用于接收手柄发来的信息，并传递给单片机，单片机也可通过接收器，向手柄发送命令，配置手柄的发送模式。

特别声明：因批次不同或者代工厂家不同，接收器的外观会有所区别，但接收器的引脚定义和解码方式是一样的，使用相同。手柄目前有一下两种品类：

① 第一种手柄上有电源灯及模式指示灯两种灯，位置位于手柄侧面，如下图所示：



② 第二种手柄上只有电源灯无模式指示灯。如下图所示：



二、接收器及转接板引脚说明

2.1 接收器硬件说明

接收器内部共有九根引脚，其中有三个引脚无实际用处故而为 NC(空)并为引出。如下图所示：



接收器引脚说明如下：

1	2	3	4	5	6	7	8	9
SDO(DAT)	SDI(COM)	NC	GND	VCC	CS	CLK	NC	NC

① SDO(DAT):接收器将遥控器的信号从此信号脚传送至主机，信号是一个 8bit 的串行数据，同步传送于时钟的下降沿。信号的读取在时钟由高到低的变化过程中完成。

② SDI(COM):主机通过此信号脚将数据传送至接收器，信号是 8bit 的串行数据，同步传送于时钟的下降沿。信号的读取在时钟由高到低的变化过程中

完成。

③ NC:空端口，无实际用处。

④ GND:电源地。

⑤ VCC:接收器工作电源，电源范围 3.3V—5V。

⑥ CS:用于提供手柄触发信号。在通讯期间，处于低电平。

⑦ CLK:时钟信号,由主机发出，用于保持数据同步；

⑧ NC:空端口，无实际用处。

⑨ NC:空端口，无实际用处。

2.2 转接板硬件说明

由于接收器的引脚不便于与主机之间连接，通过将接收器插入转接板。再通过连接转接板上与主机的信号脚传输信息。转接板上左右各 8 个引脚，共有 16 个引脚。右边的 8 个引脚的固定为固定作用，并无实际意义。左边 8 个引脚中只需要连接 SDO、SDI、GND、VCC、CS、CLK 六脚即可，CE、IRQ 无实际意义。如下图所示：



SDO、SDI、GND、VCC、CS、CLK 六脚的作用见【2.1 接收器硬件说明】。

2.3 转接板连接主机说明

转接板与主机的 SPI 信号脚相连。SPI 共 CS、SCK、MISO、MOSI4 个引脚。

官方淘宝：https://item.taobao.com/item.htm?id=653189570774

引脚作用如下所示：

① CS:设备使能信号，由主设备控制。

② SCK:时钟信号，由主设备产生。

③ MISO:主设备数据输入，从设备数据输出。信号从从设备输出至主设备。

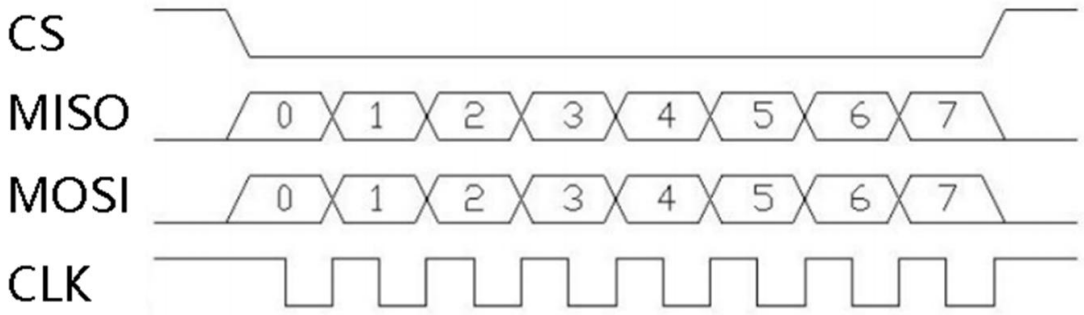
④ MOSI：主设备数据输出，从设备数据输入。信号从主设备输出至从设备。

PS2 转接板与主设备连接方式如下所示：

PS2 转接板 →	主机
VCC	VCC
GND	GND
SDO	MISO
SDI	MOSI
CS	CS
CLK	SKC

三、ps2 遥控手柄通讯协议

ps2 遥控的转接板与主设备之间使用 SPI 总线通讯。通讯时序图如下图所示：



在通讯工程中，一串数据通讯完成后 CS 才会由低转高。注：在整个通讯期间，CS 一直处于低电平转态直至通讯完成。

在时钟下降沿时，完成数据（1bit）的发送与接收是同时完成的。当单片

机想读手柄数据或向手柄发送命令时，将会拉低 CS 线电平，并发出一个命令“0x01”，手柄会回复它的 ID“0x41=绿灯模式，0x73=红灯模式”。在手柄发送 ID 的同时，单片机将传送 0x42 请求数据，随后手柄发送出 0x5A 告诉单片机“数据来了”。

一个通讯周期有 9 个字节（8 位），这些数据是依次按位传送。每个字节数据意义如下表所示，其中 idle 表示数据线空闲，无数据传输。

信号顺序	MOSI (SDI)	MISO(SDO)	Bit0、Bit1、Bit2、Bit3、Bit4、Bit5、Bit6、Bit7
1	0x01	idle[1]	
2	0x42	ID	
3	idle	0x5A	
4	WW	data	SELECT、L3、R3、START、UP、RIGHT、DOWN、LEFT
5	YY	data	L2、R2、L1、R1、△、○、×、□
6	idle	data	PSS_RX(0x00=left、0xFF=right)
7	idle	data	PSS_RY(0x00=up、0xFF=down)
8	idle	data	PSS_LX(0x00=left、0xFF=right)
9	idle	data	PSS_LY(0x00=up、0xFF=down)

举例说明：当有按键按下，对应位为“0”，其他位为“1”，例如当键“SELECT ”被按下时， Data[3]=11111110B。

当初始化设置了震动模式时，可如上表所示的 4、5 字节发送 WW、YY 来控制电机震动。WW 用于控制右侧的小震动电机，当 WW=0x00 时电极关，为其他值是小电机开。YY 用于控制左侧的大震动电机，当 YY 为 0x40~0xFF 时电机开，且 YY 值越大则电机转动越快震动越明显。

[1] idle:数据线空闲，该数据线无数据传送。



四、ps2 遥控手柄使用说明

4.1 手柄模式说明

遥控手柄的发送模式分为绿灯模式和红灯模式。

在绿灯模式（灯灭）下，手柄会发送所有按键的状态（是否被按下）信息给单片机，而摇杆则是在推到底时对应发送 UP、RIGHT、DOWN、LEFT、△、 〇、X、 □信息发送给单片机，摇杆按下时 L3、R3 有效。因此可以大致理解为绿灯模式下按键有效。

在红灯模式（灯亮）下，手柄会发送所有摇杆的数据信息（模拟值 0x00～0xFF 之间）给单片机，但不会发送按键的状态信息（L3、R3 也无效），因此在此状态下摇杆有效，所有的按键失效。

4.2 手柄的使用、连接配对说明

手柄需要两节 7 号 1.5V 的电池供电，接收器和单片机共用一个电源，电源范围为 3.3V～5V。注：不能接反，不能超压，过压和反接，都会使接收器烧坏。

手柄上有个电源开关，ON 开/OFF 关，将手柄开关打到 ON 上，在未搜索到接收器的状况下，手柄电源指示灯（绿灯）会不停闪烁。若在一定时间内接收器一直未搜索到接收器，手柄将进入待机模式，此时手柄电源指示灯将熄灭，需要通过按“START ”键，唤醒手柄。

当手柄打开，接收器供电时，手柄和接收器会自动配对，配对成功后电源指示灯（绿灯）常亮。按键“MODE”（手柄批次不同，“MODE”见也有可能标注为“ANALOG ”，但使用方法一样），可切换“红灯模式”或“绿灯模式”。指示灯如下图所示：



五、程序说明

本文以 32 例程里 ps2.c 为例讲解，主要讲解 PS2 遥控手柄的功能函数。

5.1 手柄GPIO初始化函数

端口初始化，PA6 为输入，PA4、PA5、PA7 为输出。

```
void PS2_GPIO_Init()
{
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_4|GPIO_Pin_5|GPIO_Pin_7;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure);
}
```

5.2 手柄配置函数

只需发送以下函数指令，手柄自动完成相关配置。

```
/**************************************************
Function:    void PS2_Vibration(unsigned char motor1, unsigned char motor2)
Description: 手柄震动函数,
Calls:       void PS2_Cmd(unsigned char CMD);
Input: motor1:右侧小震动电机 0x00关，其他开
       motor2:左侧大震动电机 0x40~0xFF 电机开，值越大 震动越大
**************************************************/
void PS2_Vibration(unsigned char motor1, unsigned char motor2)
{
    PS2_CS_L();
    PS2_Delay_US(100);
    PS2_Cmd(0x01);  //开始命令
    PS2_Cmd(0x42);  //请求数据
    PS2_Cmd(0X00);
    PS2_Cmd(motor1);
    PS2_Cmd(motor2);
    PS2_Cmd(0X00);
    PS2_Cmd(0X00);
    PS2_Cmd(0X00);
    PS2_Cmd(0X00);
    PS2_CS_H();
    PS2_Delay_US(100);
}

//short poll
void PS2_ShortPoll(void)
{
    PS2_CS_L();
    PS2_Delay_US(100);
    PS2_Cmd(0x01);
    PS2_Cmd(0x42);
    PS2_Cmd(0X00);
    PS2_Cmd(0x00);
    PS2_Cmd(0x00);
    PS2_CS_H();
    PS2_Delay_US(100);
}
```




```
//进入配置
void PS2_EnterConfing(void)
{
    PS2_CS_L();
    PS2_Delay_US(100);
    PS2_Cmd(0x01);
    PS2_Cmd(0x43);
    PS2_Cmd(0X00);
    PS2_Cmd(0x01);
    PS2_Cmd(0x00);
    PS2_Cmd(0X00);
    PS2_Cmd(0X00);
    PS2_Cmd(0X00);
    PS2_Cmd(0X00);
    PS2_CS_H();
    PS2_Delay_US(100);
}
//发送模式设置
void PS2_TurnOnAnalogMode(void)
{
    PS2_CS_L();
    PS2_Delay_US(100);
    PS2_Cmd(0x01);
    PS2_Cmd(0x44);
    PS2_Cmd(0X00);
    PS2_Cmd(0x01); //analog=0x01;digital=0x00  软件设置发送模式
    PS2_Cmd(0xEE); //0x03锁存设置，即不可通过按键“MODE”设置模式。
                   //0xEE不锁存软件设置，可通过按键“MODE”设置模式。
    PS2_Cmd(0X00);
    PS2_Cmd(0X00);
    PS2_Cmd(0X00);
    PS2_Cmd(0X00);
    PS2_CS_H();
    PS2_Delay_US(100);
}
```




```
//振动设置
void PS2_VibrationMode(void)
{
    PS2_CS_L();
    PS2_Delay_US(100);
    PS2_Cmd(0x01);
    PS2_Cmd(0x4D);
    PS2_Cmd(0X00);
    PS2_Cmd(0x00);
    PS2_Cmd(0X01);
    PS2_CS_H();
    PS2_Delay_US(100);
}

//完成并保存配置
void PS2_ExitConfing(void)
{
    PS2_CS_L();
    PS2_Delay_US(100);
    PS2_Cmd(0x01);
    PS2_Cmd(0x43);
    PS2_Cmd(0X00);
    PS2_Cmd(0x00);
    PS2_Cmd(0x5A);
    PS2_Cmd(0x5A);
    PS2_Cmd(0x5A);
    PS2_Cmd(0x5A);
    PS2_Cmd(0x5A);
    PS2_CS_H();
    PS2_Delay_US(100);
}
```

5.3 手柄配置初始化

手柄配置初始化，配置了开机时红绿灯模式以及禁止震动模式。其中PS2_ShortPoll()函数执行三次主要是为了建立和恢复连接。

```
void PS2_Init(void)
{
    PS2_GPIO_Init();

    PS2_SCK_H();
    PS2_CS_H();
    PS2_ShortPoll();
    PS2_ShortPoll();
    PS2_ShortPoll();
    PS2_EnterConfing();      //进入配置模式
    PS2_TurnOnAnalogMode(); //“红绿灯”配置模式，并选择是否保存
    //PS2_VibrationMode();  //开启震动模式
    PS2_ExitConfing();       //完成并保存配置
    ps2_mode_get();
}
```




5.4 发送、接收数据函数

下面两个函数分别为主机向手柄发送数据、手柄向主机发送数据。手柄向主机发送的数据缓存在数组 Data[ ]中，数组共有 9 个元素，每个元素的意义请见【三、ps2 遥控手柄通讯协议】中通信协议表（按住 ctrl+右键点击自动跳转）。

```
//向手柄发送命令
unsigned char PS2_Cmd(unsigned char CMD)
{
    unsigned char i,j=1;
    unsigned char res=0;
    for(i=0;i<=7;i++)      //逐位接收
    {
        if(CMD&0x01)
           PS2_DO_H();
        else
           PS2_DO_L();

        CMD=CMD>>1;

        PS2_Delay_US(100);
        PS2_SCK_L();
        PS2_Delay_US(100);

        if(PS2_DI())
        {
           res=res+j;
        }
        j=j<<1;

        PS2_SCK_H();
        PS2_Delay_US(1);
    }

    return res;
}
```




```
//读取手柄数据
void PS2_ReadData(void)
{
    unsigned char byte=0;
    PS2_CS_L();
    PS2_Delay_US(10);

    for(byte = 0; byte < 9; byte++)
    {
        if(byte < 2)
        {
            Data[byte] = PS2_Cmd(Comd[byte]);
        }
        else
        {
            Data[byte] = PS2_Cmd(0x00);
        }
    }
    PS2_CS_H();
    PS2_Delay_US(10);
}
```

5.5 判断手柄模式函数

此函数用于判断手柄的发送模式，也就是判断 ID 即 Data[1]的值

```
//判断是否为红灯模式,0x41=模拟绿灯，0x73=模拟红灯
//返回值；0，红灯模式
//        其他，其他模式
unsigned char ps2_mode_get(void)
{
    if( Data[1] == 0X73)
    {
        ps2_mode = PSB_REDLIGHT_MODE;
    }
    else if ( Data[1] == 0X41)
    {
        ps2_mode = PSB_GREENLIGHT_MODE;
    }
    else
    {
        ps2_mode = PSB_LOSE;
    }
    return ps2_mode;
}
```




5.6 判断按键状态函数

Data[3]与 Data[4]分别对应着 16 个按键的状态，按下为 0，未按下为 1.通过对这两个数的处理得到按键状态并返回键值。

```
//对读出来的PS2的数据进行处理,只处理按键部分
//只有一个按键按下时按下为0,  未按下为1
unsigned char ps2_key_serch()
{
    unsigned char index;
    unsigned char key_num = 0;
    //PS2_ClearData();
    PS2_ReadData();

    Handkey=(Data[4]<<8)|Data[3];      //这是16个按键  按下为0,  未按下为1
    for(index=4;index<16;index++)
    {
        if((Handkey&(1<<(MASK[index][0]-1)))==0)
        {
            MASK[index][1] = 1;
            key_num++;
        }
        else
        {
            MASK[index][1] = 0;
        }
    }
    return key_num;
}
```

5.7 摇杆值读取函数

红灯模式下，摇杆有效。摇杆的模拟值存储在 Data[5]、Data[6]、Data[7]、Data[8]中。

```
//得到一个摇杆的模拟量     范围0~256
unsigned char ps2_get_anolog_data(unsigned char button)
{
    return Data[button];
}
```